

HTML & DOM Structure for Selenium

Md. Ehsanul Haque

Senior Software Quality Assurance Engineer

Vivasoft Ltd.

Agenda

- Introduction
 - Understanding HTML
 - HTML Attributes
 - Exploring the DOM (Document Object Model)
 - Hands-on with Browser Developer Tools
 - Practical Exercise
 - Q&A / Summary
-

Introduction

- **Importance of web knowledge in automation**
- **How Selenium interacts with web pages**

Importance of Web Knowledge in Automation

- Automation tools like Selenium interact with web applications by locating and manipulating HTML elements.
- Without understanding HTML structure, attributes, and DOM, writing effective automation scripts is difficult.
- Knowledge of web elements helps in:
 - Writing stable and efficient locators.
 - Debugging element identification issues.
 - Improving test script reliability.

How Selenium Interacts with Web Pages

- Selenium communicates with the browser using WebDriver.
- It finds elements using locators (ID, Name, XPath, CSS, etc.).
- Performs actions like click, type, select based on element properties.

Understanding HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Page Title</title>
</head>
<body>
  <h1>This is a Heading</h1>
  <p>This is a paragraph.</p>
  <button id="submit-btn">Click Me</button>
</body>
</html>
```

What is HTML?

- Stands for **HyperText Markup Language**.
- It is the standard language used to create web pages.
- HTML describes the **structure and content** of a web page.
- It consists of tags that enclose content (e.g., <h1>, <p>, <div>).

Structure of an HTML Document

- **<!DOCTYPE html>**: Declares the document type.
 - **<html>**: Root of the document.
 - **<head>**: Metadata, title, styles.
 - **<body>**: Visible page content.
-

HTML Tags Based on Structure

Paired Tags (Container Tags)

- Have an opening tag (`<tag>`) and a closing tag (`</tag>`).
- Enclose content (text, other elements).

Examples:

- **`<body> </body>`** : Contains visible content
 - `<body><p>Hello</p></body>`
- **`<p> </p>`** : Paragraph text
 - `<p>This is a paragraph.</p>`

Unpaired Tags (Void / Self-Closing Tags)

- Do not require a closing tag.
- Usually self-contained (no content inside).

Examples:

- **`<br>`**: Line break
 - `<p>First line<br>Second line</p>`
- **`<img>`** Embeds an image
 - ``

Common HTML Tags and Their Roles

Tag	Purpose	Example
<h1> to <h6>	Headings of different sizes	Welcome
<p>	Paragraph text	This is a paragraph.
<a>	Hyperlink	Visit
	Display image	
<input>	Form field (text, checkbox, etc.)	<input type="text" placeholder="Enter name">
<button>	Clickable button	<button>Submit</button>
<div>	Block-level container	<div class="container">Content here</div>
	Inline container for styling	Red text

HTML Attributes

What is an HTML Attribute?

An **HTML attribute** is a special value **added to HTML elements** to provide **additional information** about them.

Note: Attributes are always specified in the opening tag and follow this syntax:

```
<tagname attribute="value">Content</tagname>
```

Purpose of Attributes

- To **configure** or **customize** the behavior of an HTML element.
- To make elements **unique** or **identifiable** (very important for Selenium).
- Some attributes affect **appearance** (like style), while others affect **functionality** (like type for inputs).

For example:

```
<input type="text" id="username" name="user"  
placeholder="Enter your name">
```

Here:

- **type:** defines the input type.
- **id:** gives a unique identifier.
- **name:** links to form data.
- **placeholder:** shows a hint inside the field.

Frequently Used Attributes in Selenium

Attribute	Purpose in Automation	Why It's Useful in Selenium	Example in HTML	Example in Selenium (Python)
id	Uniquely identifies an element on the page	Most reliable and fastest locator	<input id="username">	driver.find_element(By.ID, "username")
name	Identifies form elements, used for form submission	Reliable when id is missing	<input name="user_email">	driver.find_element(By.NAME, "user_email")
class	Groups elements for styling or JS behavior	Useful with CLASS_NAME , or in CSS/XPath locators	<button class="btn login-btn">	driver.find_element(By.CLASS_NAME, "login-btn")
type	Defines the input field behavior (text, password, checkbox, etc.)	Helps identify input type for logic or conditionally locating	<input type="password">	<i>Used within XPath or</i> CSS//input[@type='password']

Exploring the DOM

(Document Object Model)

- What is the DOM?
- DOM tree structure and hierarchy
- Relationship: parent, child, sibling, ancestor, descendant

What is the DOM?

- The **DOM** is a programming interface for HTML and XML documents.
- When a webpage is loaded, the browser converts the HTML into a **DOM tree**.
- It represents the page as a **tree-like structure** where every HTML tag is a **node** (or object).

Note: Think of the DOM as a *live map* of the webpage that JavaScript, Selenium, and other tools can interact with.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <div>
      <h1>Welcome</h1>
      <p>Hello, <strong>World</strong></p>
    </div>
  </body>
</html>
```

```
html
├─ head
│   └─ title
├─ body
│   └─ div
│       ├── h1
│       └─ p
│           └─ strong
```

DOM Relationships

Relationship	Description	Example
Parent	The immediate element above another	<code><div></code> is the parent of <code><h1></code> and <code><p></code>
Child	The element directly inside another	<code><h1></code> and <code><p></code> are children of <code><div></code>
Sibling	Elements that share the same parent	<code><h1></code> and <code><p></code> are siblings
Ancestor	Any element above (not just direct parent)	<code><html></code> , <code><body></code> , <code><div></code> are all ancestors of <code></code>
Descendant	Any element below (not just direct child)	<code><h1></code> , <code><p></code> , and <code></code> are descendants of <code><div></code>

Why This Matters in Selenium?

Understanding DOM relationships helps you:

- Helps create precise locators (especially XPath/CSS)
- Essential for dynamic element handling
- Critical for wait strategies (waiting for child elements)
- Enables relative locating (finding elements based on relationships)
- Avoid flaky locators by targeting **stable ancestor paths**
- Understand **nested elements**

Example XPath using hierarchy:

Select inside <p> inside <div>

```
driver.find_element(By.XPATH, "//div/p/strong")
```

Hands-on with Browser Developer Tools

Open Developer Tools (Chrome)

- Right click → **Inspect**
- Shortcut: Ctrl+Shift+I (Windows) / Cmd+Option+I (Mac)

Identifying Elements

- Hover to highlight sections
- Click to select an element
- View corresponding HTML in **Elements tab**

Useful Tips

- Right-click an element → Copy → **Copy selector** or **Copy XPath**
- Toggle the "**Select Element**" tool (top-left cursor icon)

Note: We'll use these selectors in Selenium to interact with elements.

Summary: Key Takeaways

- **Understanding Web Automation Starts with HTML**
 - HTML is the **structure** of every web page.
 - Selenium locates and interacts with HTML elements.
- **HTML Elements & Attributes**
 - Tags define elements (e.g., `<input>`, `<button>`).
 - Attributes like `id`, `name`, `class`, `type` help **uniquely identify elements**.

- **The DOM**
 - The **Document Object Model** is a tree representation of the page.
 - Elements have **relationships**: parent, child, sibling, ancestor, descendant.
- **Developer Tools**
 - Use **Inspect Element** to view tags and attributes.
 - Copy **CSS Selectors** and **XPaths** to build Selenium locators.
- **Locators in Selenium**
 - Good locators are **reliable**, **stable**, and **specific**.

Thank you!